

Jeevy Fabrication:

America needs dreamers and builders - we help them coordinate so they can help America.

Introduction: At Jeevy we want to make the process of building the physical infrastructure for an exciting future as simple as possible. This pamphlet describes how we plan to do this.

Cowboys and astronauts both embody a thrill for new frontiers. Their adventures necessitate invention, problem solving, and they ultimately build new things that feel like a pure form of progress. It's this romanticism that made living in a damp 16ft airstream for 6 months in rural South TX to work on launch systems for SpaceX feel like a privilege. We feel closer to the imagination engine that new things come out of when we design new things. Once something is designed, building puts us next to physical progress.

As projects become more complex it becomes necessary to have an intermediate supply chain and coordination step between design and build. This is needed for sourcing high quality builders and divy designs in a way that ensures that there is enough redundancy to build a product reliably over time. In some cases design and build engineers take on this role and it is a painful distraction. In other cases entire teams are hired that add cost, risk and communication overhead. The question then becomes :: how do we keep companies that are going to Mars or solving climate change as close to imagination and progress as possible.



The Capacity Unit:

Parts must be procured within budget, on time and within specification. Typically this is done through a bidding process where a design is circulated to several vendors. Vendors are selected so that as an aggregate they optimize for budget and quality with the constraints that they must have sufficient capacity and fulfill the functional requirements of the job. In this pamphlet we will define a new variable :: *A reliable capacity unit.*

This capacity unit is defined by a certain amount of reliable fabrication capacity in a given unit of time (i.e. 10hrs) that meets the functional requirements of a design. Reliability is defined by having above a certain quality and on-schedule delivery pass rate: say above 99% quality pass rate and above a 95% on-time delivery. The unit represents a certain amount of reserved personnel and equipment resources in a group of shops that fulfill the functional and compliance requirements demanded by the given job. All of the shops in this grouping would need to have an empirical pass and quality rate based on their historical performance. The required quality and on-time delivery rates would be achieved by developing a bench of back-up shops for a given job. Therefore, some of this capacity is direct allocation and some of it is reserve allocation to account for timeline and quality issues. The amount of capacity required for a given project therefore must account for the required lead-time and required reserve allocations.

In the world where these units are fully fungible for a given type of fabrication and quality/delivery tolerance, engineers could simply purchase the number of capacity units they

needed to complete a given job. No further coordination would be required. Jeevy's goal is to build a system that can vend these capacity units for the most complex infrastructure builds.

Unlocking the Capacity Unit:

There are 3 primary things that must go right in order to vend capacity units:

- Hub and spoke fabrication network
- AI Driven Project Management
- Standardized pricing process

Each of these will be the subject of their own pamphlets. This first pamphlet covers the hub and spoke network. Each of these are defined below.

Hub and Spoke: Defining the scope of the hub and spoke fabrication network is the main topic of this pamphlet. The network consists of fabricators across a variety of disciplines that have been onboarded and are actively using the Jeevy Gnosis ERP system. Any job awarded to that fabricator must be processed through this ERP. The following data must also be collected from each network shop:

- Number of machine and personnel resources
- Past performance on Jeevy projects
- Tracking on actively running projects
- Capacity utilization

For critical path portions of projects and last mile QA/QC a hub facility operated by Jeevy is critical. The hub only handles a fraction of total project volume but it must scale with the network. This ensures delivery of quality products over time.

Project Management: Dividing up designs between the network, the hub and managing issues that come up require an immense amount of project management and coordination. Resource allocation, or what jobs are assigned to what shops, must be dynamically adjusted as new jobs come in or assumptions change. The Jeevy Gnosis software must automatically find globally stable solutions for allocating resources in the network.

Even with very good initial allocations, issues will come up that require adjustments. For example, if a given fabricator in the network is falling behind, material must be immediately pulled and sent to the next fabricator in a rank ordered list for the given job. Excellent project managers are therefore the core of the company. For many traditional EPC's, project managers are the most expensive part of their operating expense after hourly labor. They also add the most risk because hiring, training, and managing project managers becomes exponentially harder as team sizes grow. The Jeevy Gnosis ERP provides AI driven tooling and improves situational awareness of project progress tracking in network shops that also use the software. This reduces the number of project managers for a given number of jobs by 10x.

Standardized Pricing Process: Project build price is mainly driven by labor cost, material cost, and transportation costs. Material prices can vary between vendors but by centralizing purchasing under a single Jeevy procurement umbrella, scale will equalize these differences. Transport costs can vary greatly based on how the project is being built. For

example, buying cheaper material from a vendor in a different state will increase transport costs. This optimization can be done at the project management stage. Lastly labor costs can vary regionally - this optimization can be optimized based on the exact shop network that is being used. Network shops would be incentivized to accept these projects as trust is built to guarantee that standard bidding practices are being used to provide market rate margins. Once pricing trust is established, network shops save bidding overhead which can be substantial when a 10% win rate is typical.

Model Simulations:

Set-up:

We propose a graph based simulation to verify the shop side and client side value in this model. We set up a feedforward network with configurable width and depth. Each node on a given layer represents a shop, and each layer represents one working day. The default configuration is 100 shops and 365 days.

Each shop has the following parameters:

- Capacity type: Type of fabrication. A single shop may have different capacity types.
- Num_workers: number of employees in the shop (fixed per shop type)
- quality_rate : probability that a job component passes a single quality check
- Work_efficiency: fraction of man-hours input by the shop that count as useful progress toward job completion
- Labor_cost_multiplier: multiplier of labor cost due to skill of workers
- Worker_capacity: amount of man-hours each worker can provide per day
- Mean Capacity Utilization: Percent of the shop that is occupied on average
- STD Dev Capacity Utilization: Std dev of capacity utilization
- Location: location on the map used to calculate transportation costs

Shops are randomly generated and placed at the beginning of the simulation, and are of the four following types with following parameters (true parameters are selected from a gaussian distribution with the given parameters, so no shops are identical, except num_workers which is fixed). busy_worker_rate is clipped to [0, 0.5] to ensure shops remain frequently available for allocation.

Elite (10%):

num_workers = 9

quality_rate_mean = 0.995, quality_rate_std = 0.003

work_efficiency_mean = 0.98, work_efficiency_std = 0.01

worker_capacity_mean = 10, worker_capacity_std = 1.0

labor_cost_multiplier_mean = 1.25, labor_cost_multiplier_std = 0.05

busy_worker_rate_mean = 0.15, busy_worker_rate_std = 0.02

Strong (30%):

num_workers = 6

quality_rate_mean = 0.985, quality_rate_std = 0.01
work_efficiency_mean = 0.96, work_efficiency_std = 0.02
worker_capacity_mean = 10, worker_capacity_std = 1.22
labor_cost_multiplier_mean = 1.0, labor_cost_multiplier_std = 0.1
busy_worker_rate_mean = 0.12, busy_worker_rate_std = 0.02

Average (40%):

num_workers = 5
quality_rate_mean = 0.94, quality_rate_std = 0.03
work_efficiency_mean = 0.92, work_efficiency_std = 0.03
worker_capacity_mean = 10, worker_capacity_std = 1.79
labor_cost_multiplier_mean = 0.9, labor_cost_multiplier_std = 0.15
busy_worker_rate_mean = 0.08, busy_worker_rate_std = 0.02

Risky (20%):

num_workers = 5
quality_rate_mean = 0.88, quality_rate_std = 0.05
work_efficiency_mean = 0.85, work_efficiency_std = 0.05
worker_capacity_mean = 10, worker_capacity_std = 1.79
labor_cost_multiplier_mean = 0.8, labor_cost_multiplier_std = 0.2
busy_worker_rate_mean = 0.05, busy_worker_rate_std = 0.01

Shops are all located randomly uniformly in the unit square $[0,1] \times [0,1]$.

Jobs can be introduced on any given day through the year. Each job can be split into multiple instances. For example, if a job consists of a small part being built five times, that job can be split into five components and assigned to individual shops. Each job also has a maximum number of people who can work on it. This is to set limits on the amount you can speed up a job simply by having more people work on it. For example, it is impossible to have more than a single welder work a single 2" SCH10 weld. Each job component has the following parameters:

- Deadline_days: how many days are available to complete the work
- Max_workers: maximum number of workers that can work on this job component simultaneously
- Capacity_needed: number of effective man-hours needed to complete the job
- Quality_cost: flat cost charged each time a quality check is performed on this component
- Material_cost: flat cost of job materials, charged once upon job creation
- Base_labor_rate: base labor cost per man-hour of raw input work (multiplied by the shop's labor_cost_multiplier)
- Base_transportation_cost: base coefficient scaled by distance to compute all transport costs for this component
- Max_daily_manhours_per_worker: cap on raw man-hours one worker can contribute per day

- Timeline_reliability_target: minimum work_efficiency a shop must have to be eligible for dispatch; also used to track aggregate timeline reliability
- Quality_reliability_target: minimum quality_rate a shop must have to be eligible for dispatch; also used to track aggregate quality success rates
- Delivery_location: destination coordinates used for final transportation cost calculation
- Max_delay: Delaying past this number of days means that the job will not be paid for and is marked as a failure.

All job components across a single job share the same parameters and delivery location. Jobs are sampled in proportion to their total capacity requirement (larger jobs are more likely to be selected). There are 10 job types indexed by i in $\{1, \dots, 10\}$. The following gives the formula for the parameters of each job type:

- $\text{num_components} = \text{ceiling}(i / 5)$
- $\text{deadline_days} = \text{ceiling}((6 + i*7) / \text{num_components})$
- $\text{capacity_needed (per component)} = (200 + 111*(i-1)) / \text{num_components}$
- $\text{max_workers (per component)} = 2$
- $\text{quality_cost} = 800 + 44*(i-1)$
- $\text{material_cost} = 8000 + 4444*(i-1)$
- $\text{max_daily_manhours_per_worker} = 10$
- $\text{base_labor_rate} = 8 + 4*(i-1)$
- $\text{base_transportation_cost} = 2000 + 1111*(i-1)$
- $\text{timeline_reliability_target} = 0.8 + 0.01*i$
- $\text{quality_reliability_target} = 0.94 + 0.005*i$

Delivery location is randomly assigned uniformly in the unit square, and all components of the same job share the same delivery location.

If a job component is infeasible ($\text{max_workers} * \text{max_daily_manhours_per_worker} * \text{deadline_days} < \text{capacity_needed}$), print an error and skip the entire job.

The daily man-hours a shop inputs on a component is:

$\text{daily_mh} = \min(\text{max_workers} * \text{max_daily_manhours_per_worker}, \text{max_workers} * \text{shop.worker_capacity})$

The effective progress credited toward capacity_needed each day is:

$\text{progress} = \text{daily_mh} * \text{shop.work_efficiency}$

Labor costs and shop profit are always computed on the raw input daily_mh, not the efficiency-adjusted progress.

Simulation Modes:

There are three simulation modes that will be considered:

- Base Case: Control or base case. This represents the case where 5 shops are chosen at random from the pool of shops that have sufficient capacity, and the job is given to the lowest cost option. This is represent the default part procurement motion where underlying information on the shop is unknown
- Network Exists: The second is the case where of the available shops that hit the quality and timeline requirements of a job the most cost effective option is chosen. This approach assumes that the purchaser has access to the information network
- Primary Mode - Back-ups Used: Performance of shops is known and ranked. If a shop fails a quality check then the job goes to the next available shop in the ranking. When the job goes to the next shop, 50% of the labor cost that is incurred by that quality check-point is re-applied, the part is shipped to the new location and that shipping cost is incurred, and lead time increases by 50% of the number of days elapsed before the quality check. There are zero failed quality checks.

PRIMARY MODE:

Given this, the main simulation loop operates as follows.

At each time-step we:

- 1 Perform quality checks
 - 1 Each job component has a configurable number of quality checks (default 1) evenly spaced throughout its work timeline. Check thresholds are placed at $(k / \text{num_checks}) * \text{capacity_needed man-hours of accumulated progress}$, for $k = 1$ through num_checks . The final check always lands exactly at 100% completion and is handled separately at the moment of completion (see step 5). Steps here handle only the first $(\text{num_checks} - 1)$ intermediate checks.
 - 2 Each quality check has equal probability of passing such that the probability of passing all checks combined equals the shop's quality_rate . The per-check pass probability is $\text{quality_rate}^{(1/\text{num_checks})}$.
 - 3 If a component's accumulated progress has crossed the threshold for its next intermediate check at the start of this day, the check is performed and the quality check cost is added.
 - 4 If a quality check is failed:
 - 1 If the job component duration is under max_delay , take 70% of the accumulated progress, reset the check counter, recompute check thresholds, remove the component from its current shop, and place it in the pool of components needing re-assignment. The component and job are not marked as quality failures at this stage.
 - 2 If the component is over max_dealy : mark the component and its job as quality failures. The component continues working at its current shop without re-routing.
- 2 Generate new jobs
 - 1 A random set of jobs is generated as needed and their components are added to the pool needing assignment.
 - 2 Material costs are added to each new component's total cost immediately upon creation.
- 3 Determine busy workers

1 For each shop, the number of unassigned workers occupied with external work today is drawn from $\text{Binomial}(\text{free_workers}, \text{total_capacity} * \text{capacity_utilization})$, where `free_workers` is the count of workers not currently assigned to any job component.

2 Busy workers are unavailable for new job component allocations for the remainder of this day. Workers already assigned to job components are unaffected.

4 Allocate all free job components to shops

1 For each job component in the pool needing assignment, eligible shops are identified by these criteria:

1 The shop must have at least one available worker slot after accounting for busy workers: $(\text{free_workers} - \text{busy_workers_today}) \geq 1$.

2 The shop must be able to finish the component before its deadline, accounting for the 1-day allocation delay: $(\text{remaining_progress} / (\text{daily_mh} * \text{work_efficiency})) + 1 \leq \text{days_remaining}$.

3 The shop's `quality_rate` must be $\geq$ the component's `quality_reliability_target`.

4 The shop's `work_efficiency` must be $\geq$ the component's `timeline_reliability_target`.

2 Among eligible shops, the component is assigned to the one with the lowest estimated total cost:

$\text{estimated_labor_cost} = \text{daily_mh} * \text{expected_days} * \text{base_labor_rate} * \text{labor_cost_multiplier}$

$\text{transport_cost} = \text{base_transportation_cost} * \text{distance}(\text{shop.location}, \text{delivery_location})$

3 If no eligible shop is found, the component stays in the unassigned pool and is retried the next day.

4 If a component's `days_remaining` has reached zero while in the unassigned pool, it is marked as a timeline failure and removed from the pool.

5 When a component is re-allocated after a quality failure, a transportation cost is charged: $\text{base_transportation_cost} * \text{distance}(\text{previous_shop.location}, \text{new_shop.location})$.

6 All newly allocated components undergo a 1-day delay before work begins. This day still counts against the deadline.

5 Execute daily work

1 For each job component past its allocation delay, progress is advanced: $\text{manhours_done} += \text{daily_mh} * \text{shop.work_efficiency}$.

2 When accumulated progress reaches or exceeds `capacity_needed`, the final (nth) quality check is triggered:

1 A quality check cost is added.

2 The same per-check pass probability $(\text{quality_rate}^{(1/\text{num_checks})})$ is used.

3 If failed and this is the component's first ever quality failure: halve the progress, reset the check counter, recompute thresholds, and re-route the component to the unassigned pool. The component is not yet marked as a failure.

- 4 If failed and the component has already failed a previous check: mark the component and its job as quality failures, then complete the component normally. No re-routing occurs.
- 5 If passed: mark the component as complete.
- 6 Aggregate costs
 - 1 For each active component not in allocation delay, add today's labor cost: $\text{daily_mh} * \text{base_labor_rate} * \text{labor_cost_multiplier}$. Ten percent of this is credited as profit to the shop for the day.
 - 2 For each shop, credit profit from workers busy with external work: $\text{busy_workers_today} * \text{worker_capacity} * 26.0 * \text{labor_cost_multiplier} * 0.10$, where 26.0 is the average base_labor_rate across all job types.
- 7 Handle completions
 - 1 For each component completed this day, add transportation cost from the completing shop directly to the delivery location: $\text{base_transportation_cost} * \text{distance}(\text{shop.location}, \text{delivery_location})$.
 - 2 If all components of a job are now complete, the job is marked as complete. A job is a timeline success if the minimum days_remaining across all its components is ≥ 2 at the moment the last component finishes. Otherwise it is a timeline failure.
 - 3 The job's total cost is the sum of all component total costs.
- 8 Record statistics
 - 1 For each shop, record the fraction of total worker slots occupied by active components ($\text{assigned_components} / \text{num_workers}$) and the count of busy workers today.
- 9 Decrement deadlines
 - 1 All active and unassigned components have their days_remaining decremented by 1.

Base Case Simulation

At each time-step we:

- 1 Generate new jobs if needed
 - 1 Same job pool as before, same generation method. Material costs are added immediately.
- 2 Determine busy workers
 - 1 Same binomial draw as the primary mode. Busy workers are unavailable for allocation that day and generate the same profit contribution.
- 3 Allocate new job components
 - 1 This mode allows for two different allocation methods.
 - 1 The first (random_cheapest) randomly samples up to 5 available shops with at least one available worker slot ($\text{free workers} \geq 1$) and assigns to the cheapest among them by estimated cost ($\text{estimated_labor_cost} + \text{transport_cost}$ to delivery location). Note: work_efficiency is not applied in the cost estimate in this mode.
 - 2 The second (quality_top) ranks all available shops with at least one available worker slot by quality_rate descending and assigns to the highest-quality shop.
 - 2 There is no filtering by quality_rate , work_efficiency , or deadline feasibility.

- 3 There is no re-allocation in secondary mode. Components are never moved once assigned.
- 4 The 1-day allocation delay applies as in the primary mode.
- 4 Execute daily work
 - 1 Daily work is executed the same as the primary mode: $\text{progress} = \text{daily_mh} * \text{shop.work_efficiency}$. There are no mid-work quality checks.
- 5 Aggregate costs
 - 1 Labor costs are added the same as the primary mode.
 - 2 Busy worker profit is credited the same as the primary mode.
 - 3 There are no quality check costs and no re-allocation transportation costs.
- 6 Handle completions
 - 1 Transportation cost is added directly from the completing shop to the delivery location (same formula as primary mode; no intermediate hub).
 - 2 A single final quality check is performed using the shop's raw `quality_rate` as the pass probability. If failed, the component and job are marked as quality failures. This check adds no cost and does not trigger re-routing.
 - 3 Job completion and timeline success are determined the same way as in the primary mode.
- 7 Record statistics
 - 1 Same shop statistics as the primary mode: assigned fraction, busy workers, daily profit.

STATISTICS

During the simulation, we track across all shops the following statistics:

- Fraction of total worker capacity per shop used per day (assigned to job components)
- Number of busy workers per shop per day (occupied with external work)
- Total utilization per shop per day (assigned fraction + busy fraction, clipped to [0, 1])
- Profit per day throughout year (10% of labor costs from assigned jobs plus 10% margin on busy worker external labor)
- Total profit mean across year
- Total profit variance across year
- With these statistics separated out by shop type and averaged across however many simulation runs are executed, and then plotted. The utilization plot shows assigned-worker fraction as solid lines and total utilization (assigned plus busy) as dashed lines per shop type.

Then per job we track the following statistics:

- Total cost
- Cost variance
- Quality success rate (fraction of jobs where no component accumulates two quality check failures)
- Timeline success rate (fraction of jobs completed at least 2 days before deadline)

The simulation has hyperparameters for how many 300-day runs are computed and averaged for global statistics, along with how many shops there are (default 100), how many jobs are generated each day (default 5), and how many quality checks are performed per component in the primary mode (default 1).

The Client Perspective:

The first question we want to answer is when if ever is this capacity approach worth it for a customer.

Experiment 1: How does a single job perform?

Job: Piping job, not split between different vendors, and requiring 98% quality and 95% timeline

- Capacity_type: Stainless pipe welding
- Capacity_needed: 50hrs-400hrs
- Deadline_days: 15d
- Max_split: 1
- Quality_cost: \$300
- Material_cost: \$30k
- Max_ppl: 3
- Base_labor_rate: \$100/hr
- Base_transport_cost: \$0.25cents/mile
- Location: (32.7767, -96.7969), # Dallas, TX
- Timeline_reliability_target: 0.95
- Quality_reliability_target: 0.99

Experiment 1: Run a single job at a time and vary the capacity needed and required reliability

Experiment 2: How do 20 jobs perform?

Job: Piping jobs

Run 20 jobs randomly spread across the first half of the year.

Experiment 3: How does the cost structure work?

Sweep the number of quality checks performed to see how average costs drop if you assume the base case incurs 20% of labor costs to repair failed cases

See how costs change if you reserve capacity with backup shops

The Shop Perspective:

Shops want to maximize profit. Impacts to profit include:

- Downtime - average utilization over the year
- Average margin on work
- Paying for project management/bidding overhead

Experiment 1: Run cases and calculate the profit for each shop assuming 15-25% profit margin on work the shop does itself and 10-15% on Jeevy work. Assume \$1000-\$4000 for bidding cost depending on the size of the job for external jobs - assume \$0 for Jeevy jobs.

Experiment 2: How valuable is it for elite shops for increasing profits? What happens as we scale the external utilization % of shops.

Experiment 3: What happens as we adjust quality allocation cutoffs (reduce them), how does it impact shop utilization, profits

Experiment 4: What happens if we increase quality thresholds on jobs, and consequently adjust the allowed/required number of re-allocations needed to guarantee quality

Experiment 5: How many jobs are required to meaningfully flatten the utilization curve?

Jeevy Margin

Next Paper - Shape of the Network: